

- **Phase 1 (Completed Framework):** We successfully deployed an Online/Offline Cost-Sensitive Active Learning framework for **Insurance Fraud Detection** (a classification task using highly imbalanced data). The pipeline used an XGBoost/LightGBM model and operated over a strict **10-iteration sequential batch-query loop** (sampling a batch of 2,000 points from an unlabeled pool, picking the top 100 points based on an acquisition function, querying the human oracle for true labels, appending them to the training set, and iteratively retraining).
- **Phase 2 (Current Objective):** We are pivoting completely away from fraud classification to **Insurance Pure Premium Pricing**. We are modeling the continuous expected payout (pure premium) using a **single LightGBM model configured with a native Tweedie regression objective**.

The Core Mathematical & Algorithmic Problem

1. The Mathematical Trap: Mean-Variance Coupling

The target variable $\text{pure_premium} = \frac{\text{Claim Amount}}{\text{Exposure}}$ follows a compound Poisson-Gamma (Tweedie) distribution, where a huge point mass sits exactly at zero ($\geq 90\%$ of customers never claim) and the remaining claims form a continuous, heavy right tail.

In a Tweedie distribution, the variance is explicitly linked to the predicted mean (μ) via the power law:

$$\text{Var}(Y) = \phi \mu^p \quad (\text{where } 1 < p < 2)$$

2. The Failure Mode: Calibration Collapse

If an Active Learning algorithm naively uses standard uncertainty sampling (such as ranking candidates by their predicted analytical variance or standard deviation), the math dictates that the highest-variance scores will strictly map to the largest predicted means (μ).

- **The Chain Reaction:** The active learning query strategy will exclusively pick extreme, right-tail catastrophic claims.
- **The Collapse:** The training set is immediately starved of zero-claim instances. This starves and destroys the Poisson frequency parameter (λ), stripping the model of its ability to identify low-risk policyholders. The pricing model rapidly overestimates costs for the entire population, driving up out-of-sample error.

The Two-Part Solution: Epistemic Checkpoints + SEDR

To isolate true model confusion (epistemic uncertainty) from structural tail variance (aleatoric noise) while forcing structural frequency balance, you must implement two tightly integrated actions inside the acquisition loop:

Action A: Multi-Iteration Checkpoint Tree Disagreement

Instead of looking at distribution outputs, look inside the GBDT's boosting progression. A

LightGBM model creates its final prediction via an additive sequence of trees. If early trees push a sample's prediction high but later trees violently correct it downward based on residual gradients, the model is highly uncertain about that data point regardless of whether its final premium is \$10 or \$10,000.

- **Implementation Logic:** Do not use `pred_contribs=True` (which erroneously extracts SHAP feature importances). Instead, loop through the tree iterations using a configured step size to capture true checkpoint variation:

Python

```
n_trees = model.num_trees()
step = max(1, n_trees // 50) # Sample 50 boosting snapshots to minimize computational overhead

# Extract prediction array at sequential stages of training
tree_preds = np.array(
    [model.predict(X_pool, num_iteration=k)
     for k in range(step, n_trees + 1, step)]

# Calculate Epistemic Uncertainty (Tree Disagreement Variance) per row
epistemic = np.var(tree_preds, axis=0)
```

Action B: Stratified Expected Deviance Reduction (SEDR)

To protect the model's zero-mass baseline calibration, the acquisition function must not query globally.

1. Sort the entire unlabeled candidate pool by the current model's predicted mean premium ($\hat{\mu}$) and partition it into 5 equal geometric quintiles.
2. Introduce a configurable hyperparameter `zero_mass_quota` (default initialized to 0.30). This parameter dictates that 30% of the query batch size must be filled strictly from Quintile 1 (the lowest predicted risk bounds, representing expected zero-claims). The remaining 70% of the query budget is allocated across Quintiles 2-5.
3. Within *each individual quintile*, sort candidates by their checkpoint-based tree variance ($u_{\text{epistemic}}$) and select the top candidates to satisfy that stratum's quota.

Exhaustive 3-Day Technical TODO List

Day 1: Data Ingestion, Metrics Definition & Failure Verification

- [] **Task 1.1: Environment Setup & Data Pipeline**
 - Load the standard French Motor Third-Party Liability dataset (`freemtpl2freq` and `freemtpl2sev`) via `sklearn.datasets.fetch_openml` or `sklego.datasets`.
 - Construct the continuous pure premium target: $\text{pure_premium} = \frac{\text{ClaimAmount}}{\text{Exposure}}$. Clean the data by capping extreme outliers to ensure stable convergence.

- *Assert Check*: Verify that $\geq 90\%$ of rows are strictly 0.0 and the remaining rows display a continuous, right-skewed tail. Split the data into a static validation holdout set (20%) and an Active Learning pool (80%). Create an initial training seed of 2,000 labeled instances.
- [] **Task 1.2: Code the Custom Evaluation Architecture**
 - Implement **Out-of-Sample Tweedie Deviance** as the global performance evaluator (setting baseline power parameter $p = 1.5$):

$$d(y, \mu) = 2 \left(\frac{y^{2-p}}{(1-p)(2-p)} - \frac{y\mu^{1-p}}{1-p} + \frac{\mu^{2-p}}{2-p} \right)$$
 - Implement **Zero-Mass Calibration Error (ZMCE)** to track frequency alignment:

$$\text{ZMCE} = \left| \frac{1}{N} \sum_{i=1}^N \mathbb{I}(y_i = 0) - \frac{1}{N} \sum_{i=1}^N \exp(-\hat{\lambda}_i) \right| \quad \text{where} \quad \hat{\lambda}_i = \frac{\hat{\mu}_i^{2-p}}{\phi(2-p)}$$

(Derive the model's dispersion parameter ϕ using training residuals at each iteration).
- [] **Task 1.3: Run Calibration Collapse Baselines**
 - Build a standardized 10-iteration loop that queries a batch of 100 samples per round.
 - **Pipeline Baseline 1 (Random NSL)**: Selects 100 random points each round.
 - **Pipeline Baseline 2 (Naive Variance AL)**: Scores the unlabeled pool using the native Tweedie distribution variance formula ($\text{Var}(Y) = \phi \mu^p$) and selects the top 100 highest variance points globally.
 - *Deliverable*: Plot the learning curve of ZMCE across the 10 iterations. Baseline 2 must demonstrate an exploding ZMCE curve—empirically capturing the "calibration collapse" failure mode.

Day 2: Implementing Checkpoint Disagreement & SEDR Core

- [] **Task 2.1: Code Checkpoint-Based Tree Disagreement**
 - Develop the scoring module that extracts prediction arrays at 50 sequential increments of tree models via `num_iteration=k` inside LightGBM.
 - Compute the statistical variance across these checkpoint predictions to output a clean, 1D array of `u_epistemic` values for the unlabeled pool.
- [] **Task 2.2: Program the SEDR Stratification Logic**
 - Write a structural wrapper that breaks the current pool into 5 quintiles based on predicted $\hat{\mu}$.
 - Incorporate `zero_mass_quota` as an easily tuneable hyperparameter. Construct a loop that assigns sample allocations across the quintiles (e.g., 30 samples from Quintile 1, and 17-18 samples each from Quintiles 2 through 5).
 - Extract the top `u_epistemic` candidates within each distinct quintile block until quotas are met, generating the final 100-sample query batch.
- [] **Task 2.3: Execute Benchmarking Over 4 Distinct Strategies**
 - Run the 10-iteration simulation for the remaining two channels:
 - **Pipeline Baseline 3 (Pure Tree Disagreement AL)**: Ranks the pool globally by `u_epistemic` and pulls the top 100 points without any stratification filter.

- **Pipeline Method 4 (Tree Disagreement + SEDR AL):** Your proposed combined architecture.

Day 3: Economic Utility Validation & Generalization Control

- [] **Task 3.1: Generate Lorenz and Gini Price Discrimination Metrics**
 - For the terminal iteration models of all 4 pipelines, sort the validation holdout population by their predicted pure premium on the x-axis, and map their cumulative actual losses on the y-axis.
 - Compute the Gini Coefficient for each curve to prove that SEDR delivers optimal economic price discrimination alongside statistical calibration.
- [] **Task 3.2: Controlled Synthetic Parameter Recovery Suite**
 - Construct a basic synthetic Tweedie generation helper function using known core structural inputs (μ_{true} , p_{true} , ϕ_{true}).
 - Execute Method 4 against Baseline 2 on this synthetic domain, tracking the Mean Absolute Parameter Error over training rounds to verify structural parameter convergence.
- [] **Task 3.3: Consolidate Output Matrix**
 - Aggregate performance results into a definitive comparison table documenting Out-of-Sample Tweedie Deviance, Terminal ZMCE, and Final Gini Indices across all four configurations.